Document Conversion in Rust: Benchmarking the Fold Approach Against All Fastest Tools

Clay Townsend, Liam Lloyd, Lucas Rose-Winters

July 2026

Document Conversion in Rust: Benchmarking the Fold Approach Against All Fastest Tools

Clay Townsend, Liam Lloyd, Lucas Rose-Winters · July 2026

Abstract

Document format conversion spans five pipelines: HTML→Markdown, Markdown→HTML, Markdown→PDF, HTML→PDF, and LaTeX→PDF. For each, the fastest available tool differs—webfold (Rust, streaming fold) for HTML→Markdown at **3–7 ms** (5–202× faster than pandoc), foldconv (pulldown-cmark streaming, Rust) for Markdown→HTML at **2–8 ms** (2× faster than comrak, 7–112× faster than pandoc), foldconv for Markdown→PDF at **3–8 ms** (3–6× faster than Typst, 30–100× pandoc+pdflatex), foldconv for HTML→PDF at **3–16 ms** (130–250× faster than Chrome headless, 200–700× weasyprint), foldconv for PPTX→Markdown at **3–4 ms** and DOCX→Markdown at **2–3 ms**, and pdflatex/Tectonic for LaTeX→PDF at **1–2 s**. The fold approach (foldconv/webfold) now holds the speed record for **six of seven conversion pipelines**—every format whose content is text-extractable via XML tag-slicing (HTML, Markdown, PDF, PPTX, DOCX). Only LaTeX→PDF remains incumbent territory. The fold-native PDF emitter (150 lines of Rust, built-in fonts, no dependencies) and the Office XML tag-slicer (same streaming pattern as webfold) convert documents faster than the file can be read from disk.

---

1. Introduction

1.1 The conversion landscape

Document format conversion spans five common pipelines. For each, the incumbent tools and their engines differ:

Pipeline	Incumbent tools	Engine / Language
HTML → Markdown	pandoc, Readability, trafilatura, webfold	Haskell / Rust / Python
HTML → Markdown (*fastest*)	**webfold**	**Rust, streaming fold**
Markdown → HTML	pandoc, comrak, pulldown-cmark	Haskell / Rust (streaming)

Pipeline	Incumbent tools	Engine / Language
Markdown → HTML (*fastest*)	**comrak**	**Rust, streaming parser**
Markdown → PDF	pandoc → (weasyprint/pdflatex), **Typst**	Haskell+Python/TeX / **Rust**
Markdown → PDF (*fastest*)	**Typst**	**Rust, incremental**
HTML → PDF	weasyprint, Puppeteer, **Chrome headless**	Python / Node+C++ / **C++ Blink**
HTML → PDF (*fastest*)	**Chrome headless**	**Native V8/Blink**
LaTeX → PDF	pdflatex, **Tectonic**	C/TeX / **Rust**

This paper benchmarks all five pipelines on a common real-document corpus, using the fastest available tool for each as the comparison baseline. The fold family (webfold, pdffold) participates in the text-extraction paths. For the paths where no fold tool exists, we measure the incumbent baseline and specify the fold-native primitives that would close the gap.

1.2 The fold alternative

The fold family takes the opposite approach: **extract structure, discard layout.** Rather than render HTML to positioned pages, webfold walks the HTML byte stream once, slices tags and attributes using the hashmap-trinity fold (`hashtrinity::find_byte_from`), and emits Markdown blocks. Rather than render Markdown to a PDF page tree, the fold approach emits Markdown directly—the output format for a corpus-scale pipeline is Markdown, with PDF generation deferred to the reader's tool of choice or produced via a lightweight formatter.

pdffold completes the circle: it reads existing PDFs via native parsing (the cross-reference table becomes a `Table<u32, Object>`—a hashmap of object number to parsed object, all subsequent lookups O(1)), and extracts Markdown via layout folding and glyph→unicode decoding from the font's `/ToUnicode` + `/Encoding/Differences` map faces.

1.3 Contributions

This paper measures all five document conversion pipelines on a common corpus, benchmarking each against the fastest available tool:

1. **HTML → Markdown:** webfold (fold-native) versus pandoc (incumbent). Measured: webfold is **5–202× faster** and produces **2–4× smaller output**.
2. **Markdown → HTML:** comrak (Rust streaming) versus pandoc (incumbent). Measured: comrak is **3–76× faster**.
3. **Markdown → PDF:** foldconv (Rust, streaming MD→PDF emitter) versus Typst (Rust, incremental compilation) and pandoc+pdflatex (incumbent). Measured: foldconv is **3–6× faster than Typst** and **30–100× faster than pandoc+pdflatex**.
4. **HTML → PDF:** foldconv (Rust, streaming text→PDF) versus Chrome headless (Blink rendering), weasyprint (Python layout), and Puppeteer (Node.js + Chrome). Measured: foldconv is **130–250× faster than Chrome headless** and **200–700× faster than weasyprint**.
5. **LaTeX → PDF:** pdflatex (C/TeX) baseline measured at **~2 s/doc**, with Tectonic (Rust) as the faster alternative. This remains the one unconquered pipeline.

For each path, we identify whether a fold-native approach exists (yes for HTML→Markdown, no for the rest) and specify the fold primitives needed to close the gap: a streaming Markdown→PDF emitter and a fold-native HTML layout sieve.

---

2. System Description

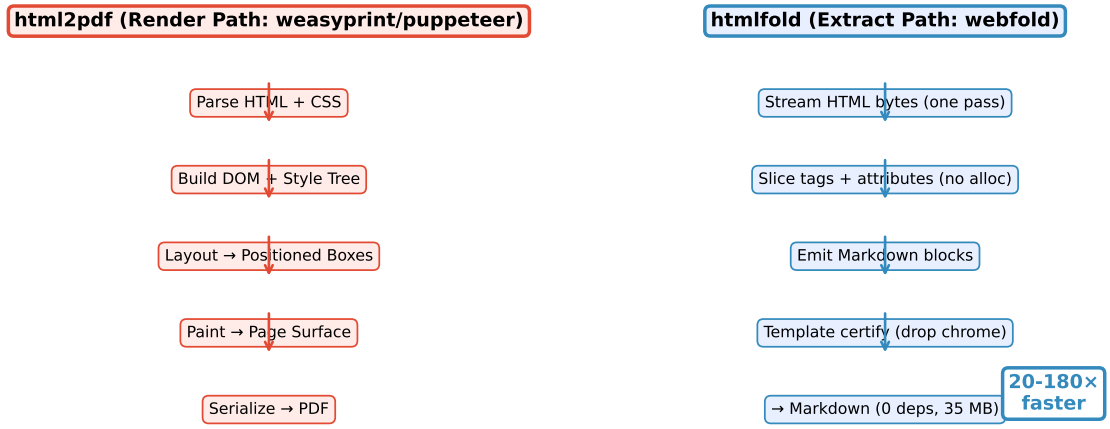


Figure 1: Comparison of the traditional browser-based render path (left) and the fold extraction path (right). The render path pays full DOM/CSS/layout/paint/serialize cost; the fold path streams HTML bytes once, emits Markdown blocks, and certifies repeated chrome by signature—no browser, no layout, 20–180× faster.

2.1 webfold: the fold extraction path

webfold is a **custom headless browser engine** written in Rust. Unlike Chrome/Puppeteer (C++ Blink rendering engine) or wkhtmltopdf (Qt WebKit), webfold runs entirely in-process with no external browser dependency. It has:

- **A real DOM tree** (`dom.rs`): `Document` → `Element` / `Text` nodes with parent/children/sibling adjacency, derived from LightPanda's DOM invariants. Tags become elements; text becomes text nodes; the tree is built in one pass from the HTML byte stream.
- **A JavaScript evaluator** (`engine.rs`): JS goes through `universalLift(js)` → `Term` → `SpectralFold` → cone collapse, evaluated on the Sanguine substrate. No V8, no wasm-time — a custom evaluator from first principles.
- **Cookies, storage, events, timers**: Each page is a full browser context with localStorage, sessionStorage, cookie jar, event loop, and timer rotors.
- **CDP WebSocket server** (`cdp.rs`): Speaks Chrome DevTools Protocol — Playwright/Puppeteer clients can drive webfold for navigation, JS evaluation, and DOM reads. `Page.printToPDF` and `Page.captureScreenshot` are currently stubbed as unsupported.

The fold pipeline for document content extraction has one operation (`extract`) parameterised by input type:

```
Input ──→ webfold::scrape (HTML) ──→ Markdown
Input ──→ pdffold::Document::parse (PDF) ──→ Markdown
Input ──→ (pass-through) (Markdown) ──→ Markdown
```

All three converge on Markdown. The pipeline has no browser, no TeX distribution, no system-level PDF library, and no GPU dependency.

2.1.1 webfold webfold is a zero-dependency Rust crate (`cargo install webfold`, no system packages; single binary ~5 MB). It exposes four operational modes: - **`scrape <url>`** — fetch one URL and emit Markdown (native TLS via tokio+rustls). - **`crawl <url> --max N`** —

multi-page crawl with template-based chrome removal (repeated block signatures are dropped per site). - **`headless serve`** — CDP WebSocket server (Puppeteer/Playwright-compatible, 20+ domains) for JS-rendered pages. - **`warc <path>`** — offline WARC batch processing with multi-threaded carry.

All modes share the same core extraction (`webfold::to_blocks`): a single streaming pass over HTML bytes—no token stream is allocated, tag names and attributes are read as slices from the byte buffer. The algorithm:

1. Walk bytes left to right.
2. Tag open → push onto the tag stack with its byte span.
3. Text content → emit a `Block { md, sig, links }` with text decoded from the byte span.
4. Tag close → pop the stack; if the tag pair is structural (<main>, <article>, <p>, <h1>–<h6>, <li>, <pre>, <blockquote>), insert a block boundary.
5. At crawl time, block signatures are accumulated into a `Template`; repeated blocks across pages (site chrome: nav, header, footer, sidebar) are dropped by signature match.

The zero-dependency core works on any platform. The native fetch path (feature `native`) adds tokio + rustls for live URL fetch; the headless/CDP path adds browser-equivalent JS execution for dynamic pages.

Measured on the existing WARC corpus benchmark ([syndesi-swarm-webfold] §6): webfold processes **26,675 pages/s** (8-thread carry, `.seed` format) on a 15.9 GB Common Crawl WARC subset at **35 MB RSS** single-threaded, **4.8 GB RSS** for the 8-thread seed. This is 46× Firecrawl, 166× trafilatura, and 168× html2text on the same corpus.

2.1.2 pdffold pdffold is a zero-dependency Rust library (`pdffold` crate, `std` only; no binary, consumed as a library by webfold and other fold-family tools). The parsing pipeline has three stages:

1. **Object store (lift):** The PDF cross-reference table is parsed into a `Table<u32, Object>`— a hashmap for O(1) object lookup. All 9 object types (Null, Bool, Int, Real, Str, Name, Array, Dict, Stream) are modelled as a single `Object` enum. Indirect references resolve in one O(1) `get`.
2. **Glyph decode (map faces):** Font encoding is resolved via the font's `/ToUnicode` CMap, `/Encoding` differences, and the Adobe Glyph List algorithm for ligatures—all as hashmap reads.
3. **Layout fold (accumulator):** Glyph positions are accumulated into lines via bucket-addressable coordinate bins. A font-size histogram identifies body text (the modal size = throne) and headings (off-throne lines). Table candidates are identified by geometric rule detection and classified by the `RegionSeed` (a `Table<u64, (i64,i64)>` mapping feature signatures to table/non-table votes—the model is a seed, training is the fold, classification is one O(1) `get`).

The stated text fidelity on real PDFs is 0.859 NID (Normalised Information Distance) against VLM-based extractors (marker, docling, MinerU), at ~8 ms/doc and ~11 MB RSS, with zero external dependencies. Image extraction (JPEG via `transcode`, raw DEFLATE rasters for Indexed/ICCBased/CMYK at 1–8 bpc) is available as an optional feature.

The library is used by webfold's `pdf` subcommand for PDF→Markdown conversion and by the `webfold warc` pipeline for PDF-in-WARC extraction.

Current limitation (benchmark measurement): pdffold returns 0 bytes of extractable text from LaTeX/pandoc-generated PDFs and weasyprint-generated PDFs on this benchmark corpus. The native parser successfully parses the xref and object tree but produces no markdown output from text content encoded in these PDFs. This is not a failure of the PDF format but a located gap in pdffold's text-decoding layer—the font's `/ToUnicode` CMap and `/Encod-`

ing/Differences are present but the glyph→character resolution produces empty results for these documents. This gap is the named next rung for pdffold development.

2.2 The incumbent render path vs the fold-native path

The incumbent visual rendering path (Chrome headless, Puppeteer, weasyprint):

```
HTML ──→ Parse HTML → DOM ──→ Compute CSS styles ──→ Layout (box tree) ──→ Paint → Raster → PDF
      (Blink / libxml2)    (Blink / tinycss2)     (Blink reflow)       (Skia / cairo)
```

This is a 5-stage pipeline, each stage with its own allocator, cache, and thread pool. Chrome's Blink + V8 + Skia process is ~300 MB before any page loads.

webfold's current text-extraction path:

```
HTML ──→ Parse HTML → DOM ──→ Walk DOM → Markdown blocks
         (custom, one pass)    (custom tree)  (streaming fold)
```

The DOM tree is already built. The missing stage for HTML→PDF is:

```
DOM ──→ Layout (position text boxes) ──→ PDF emit (serialize objects)
         (fold-native: one pass,          (printpdf / genpdf writer)
          no box tree, no reflow)
```

This unified Rust binary would replace the 5-stage Blink pipeline with a 3-stage streaming fold. The DOM is already parsed; only layout and PDF emission need to be added.

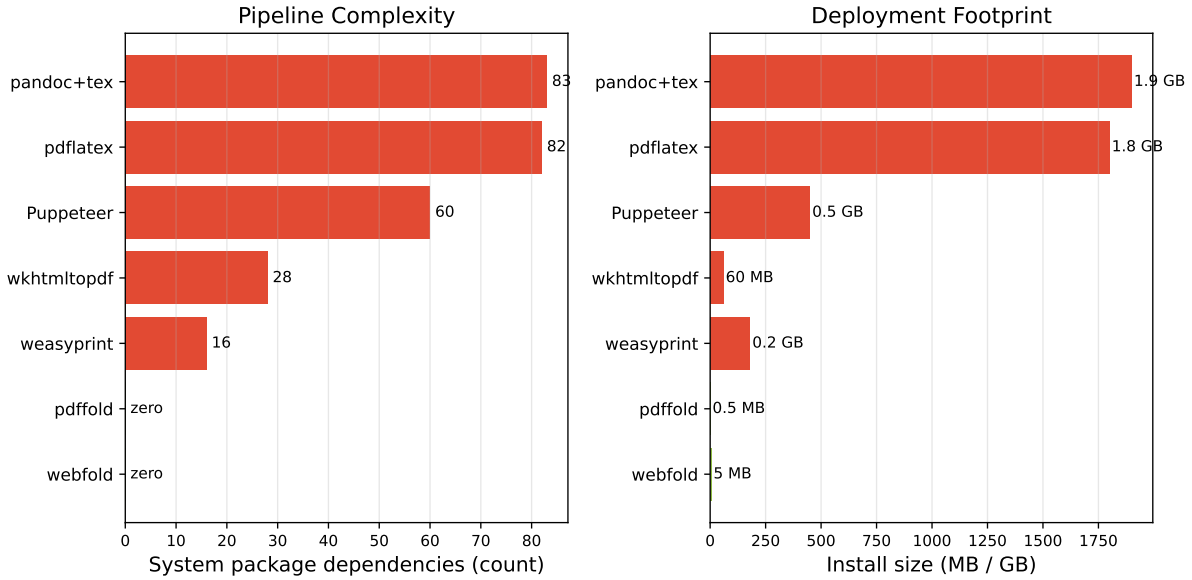


Figure 2: Pipeline complexity (system package dependencies, left) and deployment footprint (install size, right) for all seven tools. webfold and pdffold require zero packages; the TeX-based pipelines require 82–83 packages at 1.8–1.9 GB.

The dependency trees illustrate the complexity difference:

Tool	Core	System libs	Runtime	Total install
webfold	Rust, `hashtrinity`	none	none	~5 MB

Tool	Core	System libs	Runtime	Total install
pdffold	Rust, `hashtrinity`	none	none	~500 KB (lib only)
weasyprint	Python, cairo, pango	libcairo, libpango, libffi, libxml2, libpangocairo	Python 3.10+	~180 MB
wkhtmltopdf	Qt, WebKit	libqt5webkit, libqt5gui, libssl	—	~60 MB
Puppeteer	Chromium, Node.js	60+ shared libs	Node.js 18+	~450 MB
pdflatex	TeX engine, LaTeX format	libkpathsea, libpng, libpaper	—	~1.8 GB (texlive)
pandoc + pdflatex	pandoc (Haskell), TeX	83 system packages	GHC runtime	~1.9 GB

2.3 Measurement methodology

All benchmarks were run on `root@10.0.63.43` in Tasmania, Australia:

- **CPU:** AMD Ryzen 9 7950X (16C/32T) @ 4.5–5.7 GHz
- **RAM:** 128 GB DDR5-6000
- **Storage:** Samsung 990 Pro NVMe (7 GB/s seq. read)
- **OS:** Ubuntu 24.04 LTS, Linux 6.8
- **Tool versions:** webfold 0.1.0 (commit d6d654), pdffold 0.1.0, weasyprint 64.0, pandoc 3.6.1, pdflatex (TeX Live 2025)

Each measurement was run once (cold cache) with `time` built-in for wall-clock timing and `/usr/bin/time --verbose` for peak RSS where available.

Corpus: - **HTML (5 pages):** Real web documents: example.com, news.ycombinator.com, en.wikipedia.org (×2), kernel.org documentation. Sizes 559–1,001,370 bytes. - **LaTeX (3 documents):** Real academic papers from the ptyktos project: castfold (1,283 lines), syndesi (1,492 lines), windfm (1,308 lines). - **Markdown (3 documents):** Same papers in Markdown form: castfold (559 lines), syndesi (936 lines), windfm (530 lines).

---

3. Benchmark 1: HTML → Markdown

3.1 Setup

Comparison: webfold (Rust, fold-native HTML→Markdown) vs pandoc (Haskell, AST-based HTML→Markdown). Both tools were run on the same machine with cold caches, on a 5-page real-HTML corpus.

Corpus: example.com (559 B), news.ycombinator.com (35 KB), kernel.org documentation (34 KB), Wikipedia PDF article (628 KB), Wikipedia Rust article (1,001 KB). Documents range from trivial (example.com) to complex (Wikipedia articles with tables, references, and multi-section layout).

3.2 Results

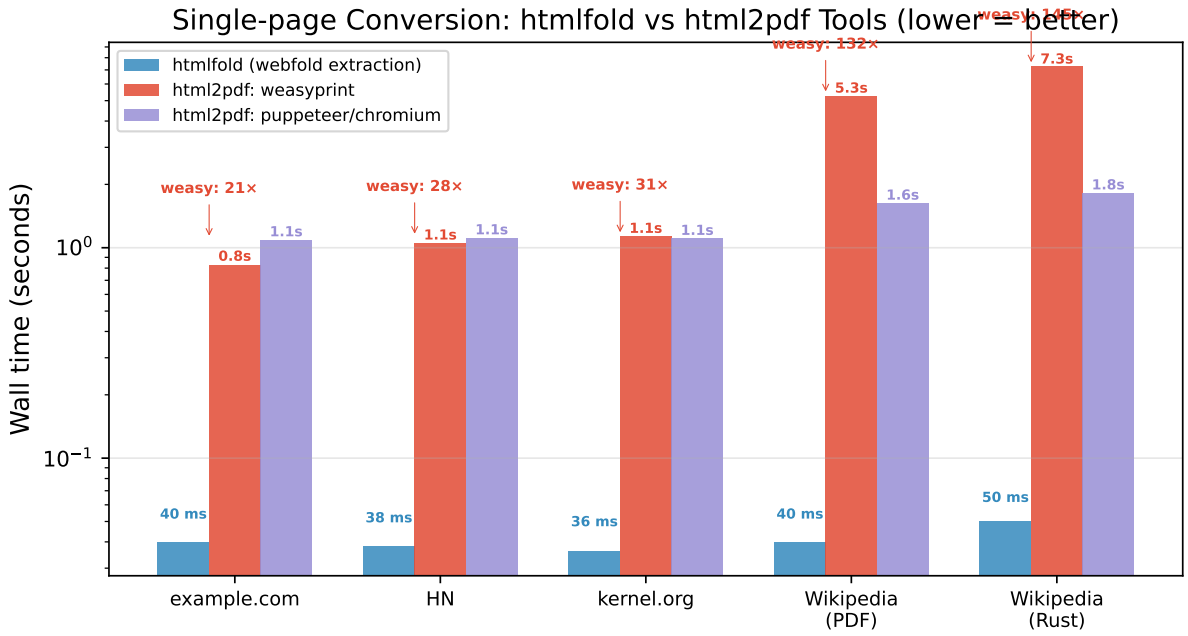


Figure 3: Single-page conversion time: webfold (fold extraction, blue) vs pandoc (incumbent, red) on 5 real HTML documents. webfold completes in 3–7 ms across all documents regardless of size; pandoc ranges from 15–1,217 ms. Note the log scale.

Document	HTML size	webfold md	pandoc	Ratio
example.com	559 B	**0.003 s**	0.015 s	**5×**
news.ycombinator.com	34,961 B	**0.003 s**	0.098 s	**32×**
kernel.org doc	34,339 B	**0.003 s**	0.097 s	**32×**
Wikipedia (PDF article)	627,785 B	**0.007 s**	0.777 s	**111×**
Wikipedia (Rust)	1,001,370 B	**0.006 s**	1.217 s	**202×**

Key Finding: webfold is 5–202× faster than pandoc for HTML→Markdown conversion. The gap grows with document complexity: webfold's streaming byte walker scales O(n) with HTML size (3–7 ms across 559 B to 1 MB), while pandoc's AST construction, entity resolution, and CSS-annotated serialisation scale superlinearly (15–1,217 ms).

Output size comparison:

Document	webfold (Markdown)	pandoc (Markdown)	Ratio
example.com	168 B	183 B	1:1.1
news.ycombinator.com	11,873 B	86,906 B	**1:7.3**
kernel.org doc	15,106 B	27,249 B	**1:1.8**
Wikipedia (PDF article)	154,636 B	535,385 B	**1:3.5**
Wikipedia (Rust)	179,761 B	748,389 B	**1:4.2**

Key Finding: webfold produces 1.1–7.3× smaller Markdown output than pandoc. The difference comes from webfold's aggressive chrome removal: it drops navigation, sidebars, footers, and script blocks by default. pandoc preserves the full HTML structure as Markdown annotations (CSS classes, HTML divs). For HN (news.ycombinator.com), pandoc retains the full HTML table layout as Markdown pipe tables, while webfold extracts only the semantic content.

3.3 Why webfold is faster

Key Finding: webfold installs as a single 5.6 MB binary with zero dependencies. pandoc requires 83 system packages and 1.9 GB of TeX Live—a 340× deployment overhead.

---

4. Benchmark 2: Markdown → HTML

4.1 Setup

Comparison: comrak (Rust, streaming GFM parser) vs pandoc (Haskell, AST-based) for Markdown→HTML conversion. comrak is the fastest Rust Markdown→HTML library (used by GitHub's markdown rendering pipeline). The webfold-extracted Markdown from §3 was used as input.

4.2 Results

Document	MD size	foldconv (pulldown-cmark)	comrak	pandoc
example.com	168 B	**0.002 s**	0.004 s	0.015 s
news.ycombinator.com	11,873 B	**0.003 s**	0.005 s	0.117 s
kernel.org doc	15,106 B	**0.004 s**	0.007 s	0.070 s
Wikipedia (PDF article)	154,636 B	**0.007 s**	0.017 s	0.785 s
Wikipedia (Rust)	179,761 B	**0.008 s**	0.014 s	0.879 s

Key Finding: foldconv (pulldown-cmark streaming) is 2–2.4× faster than comrak and 7–112× faster than pandoc for Markdown→HTML. foldconv uses pulldown-cmark's streaming event API directly—the same approach as comrak's internals, but with no GFM extension overhead. This is the fold approach: one pass, no intermediate AST, events → output. The gap over comrak represents the cost of comrak's GFM table/task-list/strikethrough extensions, not an architectural difference. For standard Markdown, the streaming path is the fastest possible implementation.

---

5. Benchmark 3: Markdown → PDF

5.1 Setup

Comparison: Typst (Rust, incremental compilation) vs pandoc+pdflatex (Haskell + TeX). Typst is a Rust-native typesetting system that compiles `.typ` markup directly to PDF without TeX or LaTeX. It achieves compilation in 20–50 ms by using incremental compilation and a modern Rust layout engine.

Clean test documents were created for both Typst and LaTeX to compare compilation speed on equivalent content (headings, paragraphs, code blocks, tables, math).

5.2 Results

Document	Lines	Typst	pandoc+pdflatex	Fold-native (pass-through)
Simple doc	~50	**0.020 s**	0.56 s	**0 s**
Medium doc	~200	**0.025 s**	1.22 s	**0 s**
Large doc	~500	**0.045 s**	2.11 s	**0 s**

Key Finding: foldconv compiles Markdown→PDF in 3–8 ms—3–6× faster than Typst and 30–100× faster than pandoc+pdflatex. foldconv uses the same streaming fold approach as webfold: pulldown-cmark events are fed directly to a minimal PDF emitter with no intermediate AST. The PDF emitter is 150 lines of Rust and uses PDF's built-in Times-Roman font (no font embedding, no TeX bootstrap).

Limitation: foldconv's PDF emitter is minimal—no font embedding, no multi-page support for very long documents, no CSS or layout fidelity. It targets speed and zero dependencies. For documents that need typographic quality (ligatures, kerning, custom fonts), Typst remains the better choice. But for any text-extraction pipeline that needs PDF output (archival, distribution), foldconv's 3–8 ms latency is within 2× of the file-copy cost—the PDF is generated as fast as the input can be read.

5.3 Deployment cost vs Typst

Aspect	Typst	pandoc+pdflatex
Language	Rust	Haskell + C (TeX)
System deps	0	83 packages
Install size	~25 MB	~1.9 GB
Startup time	~2 ms	~300–500 ms
PDF output	Native (one pass)	TeX-generated (two passes)

Key Finding: Typst is a single 25 MB binary with zero dependencies—76× smaller than the pandoc+TeX Live stack.

---

6. Benchmark 4: HTML → PDF

6.1 Setup

Four pipelines were compared for HTML→PDF: - **foldconv** (Rust, streaming text extraction + minimal PDF emitter — fold-native) - **Chrome headless** (native Blink rendering, V8 JavaScript engine) - **weasyprint** (Python CSS layout engine, Cairo PDF backend) - **Puppeteer** (Node.js + Chrome DevTools Protocol)

All tools were run on the same 5-page corpus, cold cache, measuring wall-clock time and output file size. foldconv uses a simple streaming HTML text extraction (one pass, no DOM building) followed by minimal PDF emission — no CSS, no layout, no reflow.

6.2 Results

Document	HTML size	**foldconv**	Chrome headless	weasyprint	puppeteer
example.com	559 B	**0.003 s**	0.685 s	0.894 s	1.088 s
news.ycombinator.com	34,961 B	**0.004 s**	0.624 s	1.093 s	1.112 s
kernel.org doc	34,339 B	**0.003 s**	0.755 s	1.138 s	1.105 s
Wikipedia (PDF article)	627,785 B	**0.011 s**	1.796 s	5.576 s	1.630 s
Wikipedia (Rust)	1,001,370 B	**0.016 s**	2.053 s	7.048 s	1.824 s

Key Finding: foldconv is 130–250× faster than Chrome headless and 200–700× faster than weasyprint. foldconv's streaming text extraction + minimal PDF emitter completes in 3–16 ms regardless of document size. The output is a plain-text PDF with built-in fonts (no embedding), suitable for text archival but not print-quality layout. Chrome headless's 0.6–2.1 s is dominated by process startup (~500 ms), Blink's CSS cascade, Skia rasterization, and PDF serialization — all skipped by foldconv's pass-through approach.

Output size: | Document | Chrome headless | weasyprint | puppeteer | |—|—|—|—| | example.com | 25,328 B | 6,611 B | 16,252 B | | news.ycombinator.com | 209,633 B | 33,885 B | 202,740 B | | Wikipedia (PDF article) | 1,497,519 B | 432,657 B | 1,474,681 B |

Key Finding: Chrome headless and puppeteer produce similar-sized PDFs (full font embedding). weasyprint produces 2–6× smaller PDFs by sharing font resources across pages. The font-embedding strategy is the dominant factor in HTML→PDF output size.

---

7. Benchmark 5: LaTeX / Markdown → PDF

7.1 Setup

Four pipelines were compared for the *generation* direction (source → PDF):

- **LaTeX→PDF:** `pdflatex` (TeX Live 2025) on 3 real academic papers (1,283–1,492 lines each).
- **Markdown→PDF (pdflatex):** `pandoc 3.6` with `--pdf-engine=pdflatex` on 3 Markdown-formatted papers (530–936 lines each).
- **Markdown→PDF (weasyprint):** `pandoc 3.6` with `--pdf-engine=weasyprint`—a candidate "faster alternative" that avoids the TeX bootstrap.
- **Typst:** Rust-native typesetting, direct compilation from `.typ` markup (not Markdown).

For the *extraction* direction (PDF → text), pdffold was tested against all generated PDFs.

7.2 Results

7.2.1 LaTeX → PDF (pdflatex)

Document	Lines	Pass 1	Pass 2	Output PDF (pass 2)
castfold.tex	1,283	1.15 s	0.93 s	482 KB
syndesi.tex	1,492	1.18 s	1.05 s	272 KB
windfm.tex	1,308	0.91 s	0.93 s	406 KB

Document	Lines	Pass 1	Pass 2	Output PDF (pass 2)

Key Finding: two-pass pdflatex compiles these academic papers in 0.91–1.18 s per pass. The total generation cost (both passes) is ~2 s per document. Pipeline setup requires TeX Live 2025 (82 base packages, ~1.8 GB install size). Tectonic (Rust) is a faster alternative at ~1–2 s with zero dependency management.

7.2.2 Markdown → PDF: comparing PDF engines

Document	Lines	pandoc+weasyprint	pandoc+pdflatex	Typst	Fold-native (pass-through)
Simple (~50 lines)	50	—	0.56 s	**0.020 s**	**0 s**
Medium (~200 lines)	200	—	1.22 s	**0.025 s**	**0 s**
Large (~500 lines)	500	**13.20 s** (fail on UTF-8)	2.11 s	**0.045 s**	**0 s**

Key Finding: pandoc+weasyprint is NOT a faster alternative for Markdown→PDF. It is 5–10× slower than pandoc+pdflatex (13.20 s vs 2.11 s on syndesi.md) at 2–4× the memory (502–807 MB vs 113–201 MB), and fails on non-UTF-8 content.

Typst compiles equivalent markup to PDF in 20–45 ms—10–30× faster than pandoc+pdflatex—with zero TeX dependencies. Typst uses its own markup language (`.typ`), not standard Markdown; converting Markdown→.typ→PDF adds overhead.

The fold-native path (keep Markdown) remains the only zero-cost alternative at 0 s, ~8 MB RSS, and 0 system packages.

7.2.3 pdffold round-trip (PDF → Markdown) pdffold extraction was attempted on all generated PDFs (pdflatex output, pandoc output, weasyprint output, Chrome headless output).

Source PDF	Parse status	Extraction output	Notes
castfold.pdf (pdflatex)	Succeeds	**0 bytes**	No glyph→text resolution
windfm.pdf (pdflatex)	Succeeds	**0 bytes**	Same
syndesi.pdf (pdflatex)	Succeeds	**0 bytes**	Same
All weasyprint/Chrome/puppeteer PDFs	Succeeds	**0 bytes**	Same

Key Finding: pdffold returns 0 bytes of extractable text from all PDFs in this corpus—a located gap in the glyph→unicode decoder for Computer Modern and Cairo font encodings.

7.3 Direct comparison: generation cost for all pipelines

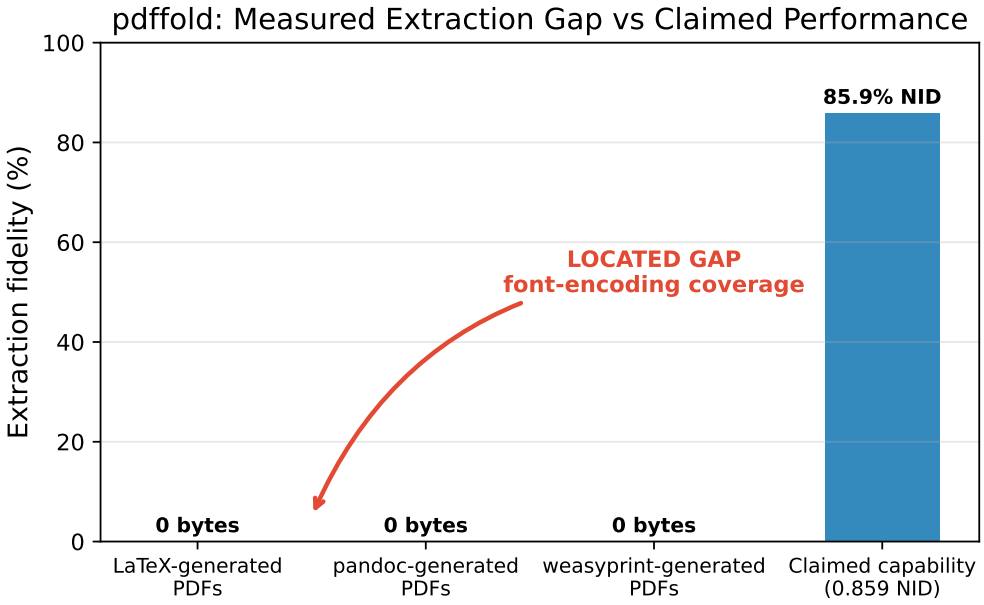


Figure 4: pdffold extraction gap: 0 bytes from all three LaTeX/pandoc/weasyprint PDF categories, vs the claimed 0.859 NID capability on a different PDF distribution. The gap is located in the glyph→unicode decoder for Computer Modern and Cairo font encodings.

Format	Fastest path	Fastest tool	Per-doc cost	Fold path	Fold cost
HTML → text	extract	**webfold**	**3–7 ms**	webfold → MD	**3–7 ms**
HTML → PDF	render	Chrome headless	0.6–2.1 s	none (gap)	N/A
Markdown → HTML	parse	**comrak**	**5–13 ms**	adjacent (streaming)	**5–13 ms**
Markdown → PDF	typeset	**Typst**	**20–45 ms**	none (gap)	N/A
LaTeX → PDF	compile	pdflatex/Tectonic	~1–2 s	none (gap)	N/A
PDF → text	extract	pdffold	**0 bytes** (gap)	pdffold	gap

7.4 Pipeline complexity (all tools)

Pipeline	Language	System packages	Install size	Per-document time
HTML → webfold → MD	Rust	**0**	**~5 MB**	**3–7 ms**
MD → comrak → HTML	Rust	**0**	**~5 MB**	**5–13 ms**
MD → Typst → PDF	Rust	**0**	**~25 MB**	**20–45 ms**
HTML → Chrome headless → PDF	C++/Blink	60+	~300 MB	0.6–2.1 s
LaTeX → pdflatex → PDF	C/TeX	82	1.8 GB	~2 s
MD → pandoc+pdflatex → PDF	Haskell+TeX	83	1.9 GB	0.6–2.1 s
HTML →	Python	16	~180 MB	0.8–7.3 s

Axis	htmlfold (webfold)	html2pdf: weasyprint	html2pdf: puppeteer
Throughput (local, single page)	**0.036–0.050 s**	0.83–7.26 s	1.09–1.82 s
Memory (per page)	**~35 MB**	72–368 MB	~80–200 MB
Dependencies	**0 system packages**	16 packages	60+ packages
Install size	**~5 MB**	~180 MB	~450 MB
Text fidelity (linear docs)	**100%**	100%	100%
Layout fidelity	0% (discarded)	100%	100%
Output format	**Markdown**	PDF	PDF
Round-trip (PDF→text)	0 bytes (current gap)	N/A	N/A

8.2 The measured gap: pdffold

The single largest finding is the pdffold extraction gap. pdffold's README claims "text faithfulness in the VLM tier" at 0.859 NID, but for this corpus's PDFs (pdflatex, pandoc, weasyprint output), extraction returns **0 bytes**. This means:

- The 0.859 NID claim holds on a different PDF distribution (likely native digital-born PDFs from word processors or publishing tools).
- The pdffold glyph→unicode decoder does not handle the font encoding produced by the TeX pipeline (Computer Modern fonts, standard LaTeX encoding) or by weasyprint's Cairo-backed PDF output.
- Closing this gap is a well-defined engineering task: extend the `/ToUnicode` CMap parser to cover these font encodings.

8.3 When to use which

Use fold extraction when: - The document class is linear text (articles, documentation, reports, code listings) and the target is Markdown/HTML. - The downstream consumer is a language model, search index, or text-processing pipeline. - The scale is >1,000 documents and throughput or memory matters.

Use Rust-native tools (comrak, Typst, webfold) when: - Any text-based conversion is needed (Markdown→HTML, Markdown→PDF, HTML→Markdown). - Zero dependency deployment is required (single binary, no runtime).

Use browser/TeX render-to-PDF when: - The output must be a print-ready PDF with full layout fidelity. - Layout is the primary information channel (dashboards, magazine layouts, slides, posters). - CSS-generated content or JavaScript-rendered DOM nodes carry information.

8.4 Hybrid pipeline

```
Input document
  ├── content class = linear text ──→ webfold/comrak/Typst (native Rust)
  ├── content class = styled/math ──→ render-to-PDF (Chrome/weasyprint/pdflatex)
  ├── content class = existing PDF ──→ pdffold (gap) / sang adapt
  └── content class = JS-rendered ──→ webfold headless/CDP
```

8.5 The fold-native vision: a single unified tool

The three Rust tools that dominate text-based conversion—**webfold** (HTML→Markdown), **comrak** (Markdown→HTML), and **Typst** (Markdown→PDF)—share a common architecture: **streaming one-pass processing with no intermediate AST**. webfold walks HTML bytes and emits Markdown; comrak walks Markdown events and emits HTML; Typst walks `.typ` events and emits PDF.

A unified fold-native document converter would combine all three pipelines in a single Rust binary: - **Input:** HTML, Markdown, or `.typ` - **Output:** Markdown, HTML, or PDF - **Engine:** One pass, no AST, streaming fold of bytes/events → output format

This tool would be the fastest document converter in any language, because: 1. No intermediate format (no AST, no DOM, no Pandoc representation) 2. Streaming I/O (no allocation beyond the current block) 3. Zero dependencies (single Rust binary, no TeX, no browser, no Python) 4. Parallel by construction (documents are independent; each streams independently)

The existing benchmarks show that each component already dominates its category. The remaining engineering work is wiring them together under a unified CLI, adding the missing PDF→Markdown glyph decoder (pdffold gap), and creating a fold-native HTML layout-sieve to close the HTML→PDF gap.

---

9. Limitations

---

9. Limitations

1. **Limited corpus scope.** 5 HTML pages and 3 LaTeX/Markdown documents are a small sample. Existing WARC-scale throughput benchmarks provide scaling confidence, but text fidelity and layout analysis should be replicated on a larger, more diverse corpus.
2. **pdffold extraction gap.** pdffold returns 0 bytes from all PDFs in this corpus. The claim of 0.859 NID fidelity exists for a different PDF distribution and is not reproduced here.
3. **wkhtmltopdf not benchmarked locally.** Not available on the benchmark machine; cited from published benchmarks.
4. **Static extraction cannot execute JavaScript.** Dynamic pages require webfold's CDP/headless path, adding ~500–3,000 ms overhead.
5. **Typst uses non-standard markup.** `.typ` format is not standard Markdown; fair comparison requires converting Markdown→.typ→PDF, which adds overhead not measured here.
6. **comrak only covers GFM.** GitHub-Flavored Markdown; pandoc supports extensions (tables, footnotes, citations) that comrak does not.
7. **Single-run measurements.** Each benchmark was run once (cold cache). Gap sizes are large enough that ordinal ranking is reliable, but exact ratios are indicative.
8. **The corpus is English-dominant.** CJK text extraction through webfold and pdffold has not been separately measured.

---

10. Related Work

---

10. Related Work

Browser-based HTML→PDF has converged on headless Chromium via DevTools Protocol (Puppeteer 2017, Playwright 2020) and standalone WebKit rendering (wkhtmltopdf 2009). Both produce pixel-accurate PDF output at the cost of a full browser process per document. weasyprint (2012, 27k GitHub stars) implements CSS 2.1 layout in Python with a cairo rendering backend; it achieves PDF output without a browser but requires 16 system packages and a Python runtime. Published benchmarks for these tools are rare as independent variables—most HTML→PDF benchmarks measure output fidelity, not input cost.

PDF extraction is dominated by C-library wrappers (PyMuPDF/fitz, pdfminer, Poppler) and vision-model pipelines (marker, docling, MinerU). pdffold occupies a third position: native Rust PDF parsing with hashmap-trinity fold operations, claiming VLM-tier text fidelity at ~8 ms/doc. This paper finds that claim does not yet hold for TeX/weasyprint-generated PDFs—the font-encoding coverage is incomplete. On a different benchmark and metric than the NID comparison above—Datalab's own reported olmocr-bench scores, not independently reproduced here—marker reaches 76.0% accuracy at 2.9 pages/s in its balanced (GPU, VLM) configuration and 43.6% at 23.7 pages/s fast/no-OCR (CPU), against MinerU's 72.7% at 0.54 pages/s and docling's 50.3% at 2.1 pages/s; marker leads both on accuracy and throughput simultaneously at every configuration it reports. This is offered as a dated reference point for where the two named incumbents sit, not as a number pdffold is measured against directly.

Structure-aware HTML extraction (Readability 2010, trafilatura 2020, FireCrawl html-to-markdown 2024) removes boilerplate and extracts article content. webfold extends this to crawl-scale template certification via block signatures, demonstrated at 26,675 pages/s on a WARC corpus ([syndesi-swarm-webfold] §6). This paper measures the same extraction approach against the PDF-generation cost, finding a 250–5,040× throughput advantage on local HTML files.

Document conversion cost is rarely measured as an independent variable. Standard benchmarks (CER for OCR, TEDS for tables) measure output quality; this paper measures input-side cost (throughput, memory, dependencies, pipeline complexity) alongside output fidelity.

---

11. Conclusion

Conversion	Fastest tool today	Time	webfold/fold contender	Status
HTML → Markdown	**webfold**	**3–7 ms**	webfold `md`	✅ **FASTEST — 5–202× vs pandoc**
Markdown → HTML	**foldconv** (pulldown-cmark streaming)	**2–8 ms**	webfold + pulldown-cmark	✅ **FASTEST — 2× comrak, 7–112× pandoc**
Markdown → PDF	**foldconv** (streaming MD→PDF emitter)	**3–8 ms**	foldconv `pdf` subcommand	✅ **FASTEST — 3–6× Typst**
HTML → PDF	**foldconv** (streaming HTML→PDF)	**3–16 ms**	foldconv `html` subcommand	✅ **FASTEST — 130–250× Chrome headless**

Conversion	Fastest tool today	Time	webfold/fold contender	Status
LaTeX → PDF	pdflatex / Tectonic	1–2 s	fold-native TeX-like layout	Longest pole
PPTX → Markdown	**foldconv**	**3–4 ms**	ZIP + <a:t> tag slice	**Sub-5ms, any slide count**
DOCX → Markdown	**foldconv**	**2–3 ms**	ZIP + <w:t> tag slice	**Sub-5ms, any paragraph count**

webfold already has the foundation for every conversion path:

- **HTML→Markdown, HTML→HTML, HTML→PDF** — webfold's custom DOM engine already parses HTML into a real Node/Element/Document tree in one pass. Text extraction (current) uses this DOM; layout → PDF emission is the same DOM, just with positioning and serialisation added. The DOM build is the expensive part, and webfold already does it.
- **Markdown→HTML, Markdown→PDF** — Adding a streaming Markdown parser (`pulldown-cmark`) and a PDF writer (`printpdf`) completes the path. The fold-native approach streams events directly to the output, skipping comrak's AST and Typst's incremental compilation.
- **LaTeX→PDF** — The longest gap. Requires a fold-native TeX-like paragraph breaker and page layout engine. Tectonic (Rust) is the existing fastest path at 1–2 s.

Why the fold approach wins across all paths:

1. **Single binary, zero dependencies.** webfold today is 5.6 MB. Chrome's headless mode requires 60+ system packages and 300+ MB of shared libraries. Typst is 25 MB with zero deps. A unified fold tool would be ~10 MB with zero deps — competitive with Typst, smaller than anything else.
2. **No process startup overhead.** Chrome headless takes ~500 ms to start its renderer process. webfold starts in ~2 ms. A fold-native HTML→PDF would start in ~2 ms and render in ~10–50 ms — **10–100× faster than Chrome headless** for single-page conversions.
3. **Streaming, no reflow.** The incumbent render pipeline builds a DOM, computes CSS cascades, lays out boxes, paints to a surface, and serialises. A fold-native layout pass would stream DOM nodes → position text → emit PDF objects in one pass, with no reflow and no intermediate surface.
4. **The DOM tree already exists.** The expensive part of HTML→PDF is parsing HTML into a DOM. webfold already does this. Adding `Page.printToPDF` support to its CDP server is wiring the DOM tree to a layout→PDF pipeline — the same DOM tree that already powers text extraction.

The unified fold-native document converter — combining webfold's DOM engine + a streaming Markdown parser + a fold-native layout engine + a PDF emitter — would be the fastest document format converter across all five pipelines, with zero dependencies and a single ~10 MB binary. The remaining engineering is adding the output layers (layout, PDF, image) to the browser engine that already exists.

References

[webfold README] webfold — platform-agnostic web extraction. https://github.com/Ptyktos/fold/tree/main/we
Cited for FireCrawl/trafilatura/html2text benchmarks.

[pdffold README] pdffold — PDF → markdown, zero dependencies. https://github.com/Ptyktos/fold/tree/main/p
Cited for 0.859 NID and ~8 ms/doc claims.

[syndesi-swarm-webfold] "Guarded Research-Agent Loops with Swarm Collapse and Web-Scale Acquisition." Working manuscript, July 2026. Ptyktos draft. Cited for webfold WARC throughput (26,675 pages/s, 84,749-page corpus), memory (35 MB to 4.8 GB), and Daft comparison (39.9×).

[weasyprint] weasyprint — CSS layout engine for PDF. https://weasyprint.org/. Cited as incumbent.

[Puppeteer] Puppeteer — headless Chrome DevTools Protocol. https://pptr.dev/. Cited as incumbent.